Simplify interfaces before shrinking certificates

A further Rocq and Lean proof review for ProveIt
21 October 2026

Recommendation. First extract one calculus-independent enumeration/fairness theorem in each assistant. Next separate the decoder's finite route and clock from its tape semantics. For the newly supplied affine lane, replace bespoke numerical halting budgets with a reusable reachable-suffix induction principle. These simplify repeated arguments without changing a machine, calculus, or public consistency statement.

Pinned review. This report concerns main at `6f0d42d368506bc29c70ace1c992ed88b2651346` (23:28 UTC). It is a source review and ordinary mathematical proposal, not a new kernel build. No incoming executable was run. No source patch was committed.

1. What is already done, and what is not

The recent Lean changes must not be proposed again:

- Phase eight shares implication and seed semantics.
- The phase-nine return/prefix work already constructs decoder chains from source paths and provides exact arbitrary-gap/register prefix execution.
- Commit `d93adc41745b` replaces the generic implication core by a 58-step condensed-detachment derivation. The latest merge retains that core; its Nat adapter is unchanged.

The maintained Rocq files and the receipts imported from other branches are different evidence boundaries. The latest merge explicitly retains the older checked Rocq replay. A recorded normalized-source recursive check at `9bac1c40` must not be attributed to different source bytes merely because its receipts occur in the history.

The newest incoming packet supplies a checked 367-control affine source lane and CM215 integration evidence. It explicitly says that generated graph/rank/location sources (284,987 bytes) exceed the earlier normalized certificate (109,958 bytes). Thus fewer graph vertices are not by themselves a shorter proof or a better maintained default. Its seed cleanup and CM215 design are inherited, not new discoveries of this review.

Scope of improvement. The targets below concern proof organization and reusable lemmas. No wall-clock, memory, compiled-proof-size saving, or smaller-than-215-state machine is established here. Exact savings require an implementation and a pinned rebuild.

2. First priority: one enumeration/fairness core

Lean's `Enum222`, `EnumDom`, and `EnumPerm` repeat the same scheduler argument around different command semantics. In Rocq, `L1_CompPerm` and `SeedPerm` repeat candidate-index reachability and halting transport. The natural abstraction is the scheduler, not the entire proof calculus.

Let S be states, $\delta : S \to \mathrm{option}(S)$ the step function, and $p, n : S \to \mathbb{N}$ the program remainder and next candidate index. Require only these successful-step laws:

$$\delta(s) = \mathrm{Some}(t),\ p(s) > 0 \Longrightarrow p(t) < p(s)\ \wedge\ n(t) = n(s),$$

$$\delta(s) = \mathrm{Some}(t),\ p(s) = 0 \Longrightarrow n(t) = n(s) + 1.$$

No uniform bound on $p(t)$ is needed at a boundary step. The new rank is a natural number and can be drained separately.

Proposition 1 (Drain and cofinal boundary reachability)**.** *Every state either reaches a halting state or reaches a boundary $p = 0$ without changing its candidate index. Starting at a boundary s_0, for each $k \in \mathbb{N}$, either the machine halts or it reaches a boundary s_k with $n(s_k) = n(s_0) + k$.*

Proof. For the first claim use strong induction on $p(s)$. Rank zero is already a boundary. Otherwise inspect $\delta(s)$: failure gives halting, while success strictly decreases rank and preserves the index. For the second claim induct on k. At a reached boundary inspect one step, retaining the possibility of halting; otherwise the index increments once, and the first claim supplies the next boundary. Compose the finite runs. This proof does not inspect an axiom, formula code, theorem stack, or command decoder. □

Completeness interface. Suppose inconsistency supplies, for every lower bound N, an encoded command list of code $M \geq N$ that halts from *every* payload state. Boundary reachability reaches candidate M (or has already halted), and the command-list execution lemma finishes. Arbitrary initial payload is essential here: soundness may require an invariant, but this completeness argument does not.

Concrete refactor.

1. Put generic run composition, drain and cofinal reachability in one small module in each language. Keep concrete definitions and their old public theorem names as wrappers initially. These are maintenance-wide savings; the current K215 endpoint need not import every older enumerator.
2. In Lean instantiate the scheduler for all three enumerators. Retain the variant-specific `step_pl`/`step_start` or `step_eq`/`decode_le` adapters, command execution and axiom contracts.
3. In Rocq generalize `reach` to a boundary seed with arbitrary initial index; make `reach_from` a wrapper. Unify `halts_from_reach` and `halts_from_reached`. A single cofinal completeness theorem then specializes to the ordinary and seeded enumerators.

The padded candidate construction must remain in the concrete adapter: prove its code is at least the starting index and that its execution contract holds for all payloads. Do not replace this by a vague assertion that enumeration is fair. Preserve early-halting disjunctions, including on malformed codes; do not quietly add a well-formed-input assumption.

3. Lean: choose the decoder route before the tape

The new `KDecoderPrefix215.entry_uniform` first calls `entry_run` at gap 0 and register list [] to select a port. For an arbitrary gap/register list it calls the execution theorem again, then aligns the results through `routed_control_unique`, `rolePath_unique` and `snoc_unique`. This is correct, but it derives a syntactic fact through two semantic executions.

Instead strengthen the source-path result to the following quantifier order:

$$\exists\, \mathrm{route}, q, b, \mathrm{op}, \mathrm{arg}, k, \quad \forall a, t, \quad \mathrm{Before}(a, t) \wedge \mathrm{Reads}(a, t, w) \Longrightarrow \mathrm{run}^k(84, a, t) = (q, a + |w| - 1, t).$$

Here the route, entry port and clock depend on the source path and checked block data, not on t. They may be a dependent record or a pure existential; an executable route function is not necessary for the first refactor.

Construction and proof. Split the existing source-path induction into two parts:

1. Construct a finite routed block chain using the source path, role lookup and checked edges. The primitive case contributes zero internal decoder steps. At an internal node add the chosen block's fixed step count to the child's count.
2. Prove the route sound for every tape satisfying its guards. Each local block restores the tape and advances the head as specified, so the prefix-reading and short-history guards pass to the next block. Compose existing framed-window block correctness.

Then the zipper bridge is applied once parametrically in the gap and registers. `entry_uniform` becomes a projection of this stronger statement: no sample execution, second existential selection, or port-identification transport is needed. Moreover the exact same clock k works for all those payloads, stronger than the current per-payload existential clock.

Guardrails. The whole source path determines the role; a physical state or two-bit suffix does not identify every role. Keep the terminal source bit and full routed-entry witness. The clock stops at primitive entry: it is not a fixed cost for primitive execution or the return scan. Do not conclude this strengthening from the current abstract `FullEntry` interface alone, which has tape-dependent existentials. Prove it from the checked route construction, then derive `FullEntry` for compatibility. Remove uniqueness helpers only after a caller census. A route in a computational type must be built from decidable data or explicit indices, not by forbidden elimination of a general proposition-valued path proof into a natural-number clock.

4. Rocq: avoid a second numerical halting semantics

The incoming affine lane's `CoreContraction.v` introduces `within`, a bounded halting predicate, solely to justify completeness after normalizing several transitions. It then develops budget raising, step and multi-step cancellation, normalization preservation and conversion from halting to a budget.

A reusable induction principle can express exactly what is needed. Write $s \to^* t$ for finite reachability and $H(s)$ for inductively defined halting. For an ordinary deterministic transition relation, assume halting states have no successors. Let P satisfy

$$\mathrm{Halt}(s) \Rightarrow P(s), \qquad s \to t \ \wedge \big(\forall u,\ t \to^* u \Rightarrow P(u)\big) \Rightarrow P(s).$$

Proposition 2 (Reachable-suffix halting induction)**.** *Under these assumptions,* $H(s) \Rightarrow P(s)$.

Proof. Strengthen the induction on the derivation of $H(s)$ to $\forall u,\ s \to^* u \Rightarrow P(u)$. At a halted state every reachable suffix is the state itself, since no first step exists. At a step $s \to t$, the induction hypothesis supplies P on every suffix of t, hence the step premise supplies $P(s)$. A nonempty path from s has the same first successor by determinism and is therefore covered by the induction hypothesis; the empty path uses $P(s)$. □

For the concrete register semantics, determinism is only up to `req`, pointwise register equality. Strengthen the conclusion over all reachable endpoints. At a zero-step suffix, transport the original first edge to the extensionally equal register file using `step_source_ext`; at a nonempty suffix transport its first successor using determinism up to `req`. No extra extensionality premise on P is needed. The zero-step constructor already permits `req`. Do not replace pointwise equality by equality of functions or add functional extensionality.

Application to the core. Prove once that normalization is a reachable state:

$$\exists m, \quad s \to^m \text{normalize}(f, s).$$

Induct on fuel; each recognized drain uses the existing `drain_exact_steps`, and each unrecognized node or exhausted fuel is a zero-step result. Instantiate P by `core_halts`. Its step constructor asks for P at the normalized successor, supplied directly by the reachable-suffix premise. The strengthened induction handles `req` through source transport rather than a new equality axiom.

This replaces the `within` family, `accelerate_within`, `normalize_within`, `halts_has_budget` and the separate budget induction. Soundness still expands the exact finite macro path. It does not need the number of contracted steps, a rank certificate, or a claim that normalization always finds a retained vertex. Finite-core membership remains a separate checked fact.

This is an ordinary proof proposal, not a compiled replacement. The helper theorem has an initial cost; measure total maintained proof size after both this application and other deterministic macro proofs use it. The source-to-graph bridge is a possible later client, but its cross-semantics correspondence obligations must not be removed merely by analogy.

5. Acceptance plan and priorities

1. Implement scheduler extraction first, keeping concrete state/step functions definitionally unchanged. Check all three Lean enumerator clients and both Rocq completeness specializations. Preserve public theorem types.
2. Add the stronger tape-independent decoder-route theorem. Derive the old prefix interface as a corollary; bind the same literal table, source tree and role data. Check the full prefix-to-zipper clients.
3. Prototype reachable-suffix induction in the auxiliary affine lane. Keep old/new public equivalence statements side by side until both compile with the same assumptions.
4. Rebuild the exact affected dependency closure, print endpoint assumptions and run recursive Rocq checking on the exact final sources. Record fresh versus reused modules. Maintain generator/source byte bindings wherever generated modules change.
5. Compare handwritten proof lines, generated bytes, peak memory, compilation and kernel-check time separately. A shorter statement, fewer graph vertices, or smaller compressed archive does not establish all these improvements.

Bottom line. The most conservative common improvement is to prove scheduling once. The highest-leverage new Lean interface is a tape-independent decoder certificate with a fixed internal clock. The affine Rocq lane offers a clean test case for replacing explicit halting budgets with a general suffix-closed induction rule. None requires a new axiomatization or a machine-state reduction.

Pinned sources

All paths are relative to ProveIt at review cutoff 6f0d42d36850.

- Lean: `Computability/BusyBeaver/Lean/BusyBeaver/ZFC/`: `Enum222.lean`, `EnumDom.lean`, `EnumPerm.lean`, `KDecoderPrefix215.lean`, `DecoderPaths.lean`, `DominatedImplication.lean`.
- Rocq: `Computability/BusyBeaver/Research/ZFC/k225-formal/rocqperm/`: `L1_CompPerm.v`, `SeedPerm.v`.
- Incoming: checked affine core and CM215 packet; `affine_core/CoreContraction.v`, `DrainCore.v`, `SourceGraphBridge.v`.
- History: shared return and decoder prefixes; 58-step generic implication; merge boundary.